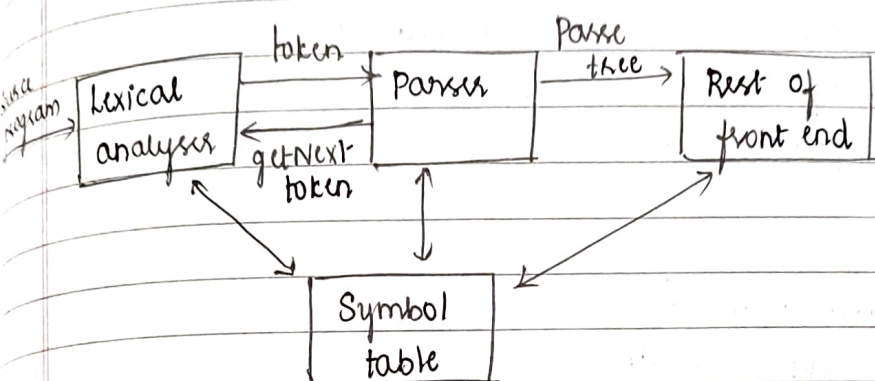


Choice - 2 & 3.

24.2.25

Module - 02.Syntax Analysis.

* Passing: It is the process of getting tokens from the lexical analyser and obtains a derivation for the sequence of tokens & builds a parse tree.



3 types of parsers:

1. Universal
2. Top-down
3. Bottom-up

Syntax error handling.

1. Lexical errors.
2. Synthetic errors.
3. Semantic errors.
4. Logical errors.

Context free grammar:

Defn: The context free grammar is 4 tuple

$$G = (V, T, P, S) \text{ where}$$

V - is set of variables, also called non-terminals.

T - is set of terminals.

P - is set of productions. All productions are of the form $A \rightarrow \alpha$ where A is a non-terminal and α is string of grammar symbols.

S - is the start symbol.

Eg: $\text{stmt} \rightarrow \text{if}(\text{expr}) \text{stmt} \text{ else } \text{stmt}.$

if & else - terminals & '(' & ')'

stmt & expr - non terminals.

→ Notational conventions.

1. Terminals:

- lowercase letters - a, b, c, d etc.
- operators - +, *, -, /
- punctuation - .

2. Non-terminals:

- uppercase letters.
- S - start symbol.
- lowercase, italic - expr or stmt .

⇒ Derivation: The process of obtaining string of terminals and/or non-terminals from the start symbol by applying some set of production is called derivation.

1. Leftmost derivation.

2. Rightmost derivation.

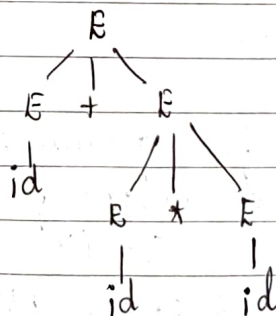
$$E = E + E$$

$$= id + E$$

$$= id + E * E$$

$$= id + id * E$$

$$= id + id * id$$



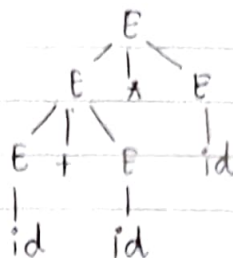
⇒ Sentinals:

1. Left Sentinal form.

2. Right

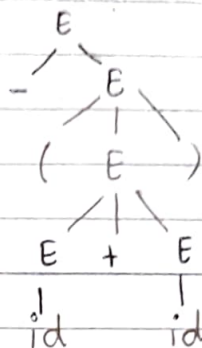
⇒ Parse tree : is a graphical representatⁿ of a derivatⁿ that filters out the order in which productions are applied to replace non-terminals.

$E \rightarrow E * E$
 $\rightarrow E + E * E$
 $\rightarrow id + E * E$
 $\rightarrow id + id * E$
 $\rightarrow id + id * id$



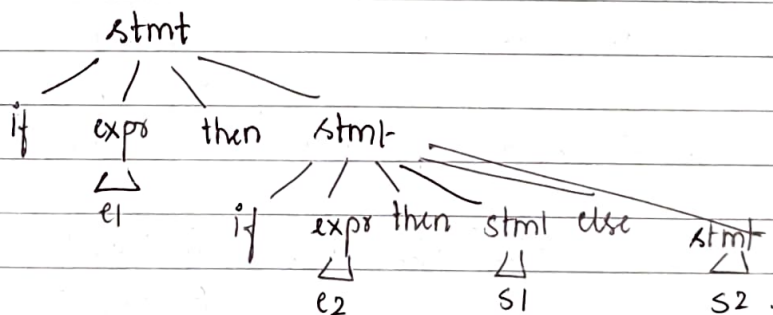
String : $-(id + id)$

$E \rightarrow -E \rightarrow -(E)$
 $\rightarrow -(E + E)$
 $\rightarrow -(id + E)$
 $\rightarrow -(id + id)$



⇒ Ambiguity : A grammar that produces more than one parse tree for some sentence is said to be ambiguous.

⇒ Eliminating ambiguity : An ambiguous grammar can be rewritten to eliminate the ambiguity.



Imp ⇒ Elimination of left recursion :

A grammar is left-recursive if it has a non-terminal A such that there is a derivation $A \xRightarrow{+} A\alpha$.

* Top down parsing methods can't handle left-recursive grammars.

Rule : $A \rightarrow \alpha / \beta$

We may replace it like : $A \rightarrow \beta A'$
 $A' \rightarrow \alpha A' / \epsilon$

1) $T \rightarrow T * F / F$

LR : $T \rightarrow FT'$

$T' \rightarrow *FT' / \epsilon$

2) $E \rightarrow E + T / T$

$T \rightarrow T * F / F$

$F \rightarrow (E) / id$

$E \rightarrow TE'$

$E' \rightarrow +TE' / \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' / \epsilon$

$F \rightarrow (E) / id$

3) $S \rightarrow aB / ac / sd / Se$

$B \rightarrow aBc / f$

$c \rightarrow g$

(Algorithm is important).

$S \rightarrow ~~abf~~ aBS' / acS'$

$S' \rightarrow ds' / es' / \epsilon$

$B \rightarrow aBc / f$

$G \rightarrow g$

$\Rightarrow$ Left factoring : is a grammar transformation that is useful for producing a grammar suitable for predictive or top-down parsing.

Rule : $A \rightarrow \alpha\beta_1 / \alpha\beta_2$ then $A \rightarrow \alpha A'$
 $A' \rightarrow \beta_1 / \beta_2$

Eg : $S \rightarrow iEtS / iETSeS / a$

$E \rightarrow b$

$\Rightarrow S \rightarrow iEtSS' / a$

$S' \rightarrow \epsilon / es$

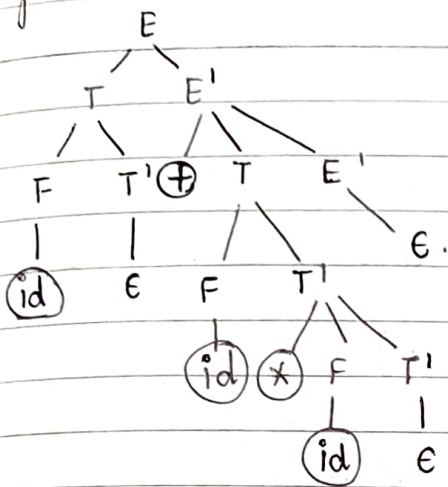
$E \rightarrow b$

$$S \rightarrow SS + / SS * | a \Rightarrow \begin{array}{l} S \rightarrow SSSS' / a \\ S' \rightarrow + / * \end{array}$$

Top down parsing:
top-down parser

→ A top-down parser tries to create a parse tree from the root towards the leaf scanning input from left to right -

tree :



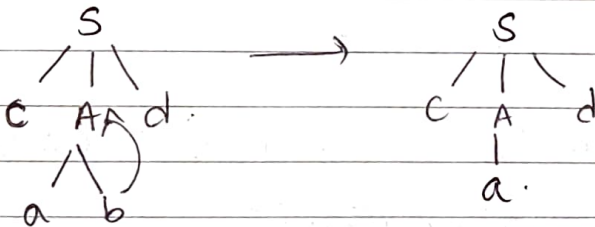
first, check for left recursive
& then go for parse tree.

⇒ Recursive decent parsing: If the given string doesn't match, then we do backtracking.

$$S \rightarrow CA_d$$

String : cad.

$$A \rightarrow ab/a$$

~~3 non-terminal (must)~~

⇒ First and follow: (check for left recursive)

$$E \rightarrow TE'$$

First- $(E) : \{ (, id \}$

$$E' \rightarrow +TE' / \epsilon$$

First (E') : $\{ +, \epsilon \}$

$$T \rightarrow FTI$$

First (τ) : $\{ (, id) \}$

$$T' \rightarrow *FT' / \epsilon$$

First (T_1) : $\{*, \epsilon\}$

$$F \rightarrow (E) / \text{id}$$

First (F) : {c, id}

↓
gt should have terminal.

Rules:

1. FOLLOW(S) always contain \$ (end of input)
2. If a non-terminal A is followed by a terminal b in a production $A \rightarrow \alpha b \beta$ then $b \in \text{FOLLOW}(A)$.
3. If a non-terminal A is followed by a non-terminal B, then $\text{FIRST}(B) \subseteq \text{FOLLOW}(A)$ (except ϵ)
4. If $\epsilon \in \text{FIRST}(B)$, then $\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$.

$$\rightarrow \text{FOLLOW}(E) = \{ \$,) \}$$

$$\rightarrow \text{FOLLOW}(E') = \{ \text{follow}(E) \} = \{ \$,) \}$$

$E \rightarrow T(E)$ Here it is ϵ $\therefore E'$ is not following.

$$\rightarrow \text{FOLLOW}(T) = \{ \text{first}(E') \}$$

$$E \rightarrow T(E') \text{ (Here T is following } E', \text{ which is a non-terminal)}$$

$$= \{ +, \epsilon \}$$

Here if we encounter ϵ then

$$= \{ +, \text{follow}(E') \} = \{ +, \$,) \}$$

$$\rightarrow \text{FOLLOW}(T') = \{ \text{follow}(T) \} = \{ +, \$,) \}$$

$$T \rightarrow F(T')\epsilon$$

$$\rightarrow \text{FOLLOW}(F) = \{ \text{first}(T') \} = \{ *, \epsilon \} = \{ *, \text{follow}(T') \}$$

$$T' \rightarrow *(F')/\epsilon = \{ *, +, \$,) \}$$

$$\begin{aligned} 4) \quad D &\rightarrow L; T \\ L &\rightarrow L; id/id \\ T &\rightarrow int/real \end{aligned}$$

Do left recursion

$$\begin{aligned} D &\rightarrow L; T \\ L &\rightarrow L L' \\ L' &\rightarrow ; id L' / \epsilon \\ T &\rightarrow int/real \end{aligned}$$

$$\begin{aligned} \text{First}(D) &= \{ id \} \\ \text{First}(L) &= \{ id \} \end{aligned}$$

$$\text{First}(D) = \{ id \}$$

$$\text{First}(L) = \{ id \}$$

$$\text{First}(T) = \{ int, real \}$$

$$\text{Follow}(D) = \{ \$ \}$$

$$\text{Follow}(L) = \{ ; \}$$

$$\text{Follow}(T) = \{ \text{follow}(D) \} = \{ \$ \}$$

$$3) \quad E \rightarrow E+T/T$$

$$T \rightarrow TF/F$$

$$F \rightarrow F*/a/b$$

Left recursive: $E \rightarrow TE'$

$$E' \rightarrow +TE'/\epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow FT'/\epsilon$$

$$F \rightarrow aF'/bF'$$

$$F' \rightarrow *F'/\epsilon$$

$$\text{First}(E) = \{ a, b \}$$

$$\rightarrow \text{Follow}(E) = \{ \$ \}$$

$$\text{First}(E') = \{ +, \epsilon \}$$

$$\rightarrow \text{Follow}(E') = \{ \text{follow}(E) \} = \{ \$ \}$$

$$\text{First}(T) = \{ a, b \}$$

$$\rightarrow \text{Follow}(T) = \{ \text{first}(E') \} = \{ +, \epsilon \}$$

$$\text{First}(T') = \{ a, b, \epsilon \}$$

$$= \{ +, \text{follow}(E') \} = \{ +, \$ \}$$

$$\text{First}(F) = \{ a, b \}$$

$$\rightarrow \text{Follow}(T') = \{ \text{follow}(T) \} = \{ +, \$ \}$$

$$\text{First}(F') = \{ *, \epsilon \}$$

$$\rightarrow \text{Follow}(F) = \{ \text{first}(T') \} = \{ a, b, \epsilon \}$$

$$= \{ a, b, \text{follow}(T') \} = \{ a, b, +, \$ \}$$

$$\rightarrow \text{Follow}(F') = \{ \text{follow}(F) \} =$$

$$= \{ a, b, +, \$ \}$$

$$\text{First}(D) = \{ id \}$$

$$\text{Follow}(D) = \{ \$ \}$$

$$\text{First}(L) = \{ id \}$$

$$\text{Follow}(L) = \{ ; \}$$

$$\text{First}(L') = \{ ;, \epsilon \}$$

$$\text{Follow}(L') = \{ \text{follow}(L) \} = \{ ; \}$$

$$\text{First}(T) = \{ int, real \}$$

$$\text{Follow}(T) = \{ \text{follow}(D) \} = \{ \$ \}$$

$$4) \begin{aligned} S &\rightarrow TS \mid [S]S \mid S \mid \epsilon \\ T &\rightarrow [X] \\ X &\rightarrow TX \mid [X]X \mid \epsilon \end{aligned}$$

$$\begin{aligned} \text{First}(S) &\rightarrow \{ [,), \epsilon \} & \text{Follow}(S) &= \{ \$,] \} \\ \text{First}(T) &= \{ [\} & \text{Follow}(T) &= \{ \text{first}(S) \} \\ & & &= \{ [,), \epsilon \} \\ \text{First}(X) &= \{ [, [\epsilon \} & & \\ & & &= \{ [,), \text{follow}(S) \} = \{ [,), \epsilon, \$ \} \\ \text{Follow}(T) &= \{ [,), \epsilon, \$ \} \\ \text{Follow}(X) &= \{] \} \end{aligned}$$

$$\begin{aligned} \text{Follow}(S) &= \{ \$,] \} \\ \text{Follow}(T) &= \{ \text{first}(S) \} = \{ [,), \epsilon \} = \{ [,), \epsilon, \text{follow}(S) \} \\ &= \{ [,), (\$,] \} \cup \text{first}(X) = \{ [,), (\$,], \\ \text{Follow}(X) &= \{] \} \\ \text{Follow}(T) &= \{ \text{first}(S) \} = \{ [,), (\epsilon \} = \{ [,), (\$,] \} \\ &\quad \cup \quad \text{Follow}(X) \quad \cup \\ &= \{ \text{first}(X) \} = \{ ([, [\epsilon \} = \{ ([, [], \\ &= \{ [,), (\$,] \} \end{aligned}$$

⇒ LL(1) grammar (Predictive parsing table):
* first L - scanning input from left to right.
* second L - leftmost derivation.

Step 1: Remove left recursive.

$$\begin{aligned} 1) \quad E &\rightarrow E+T/T \Rightarrow \text{left recursive} \\ T &\rightarrow T*F/F & E &\rightarrow TE' \\ F &\rightarrow (E)/id & E' &\rightarrow +TE'/\epsilon \\ & & T &\rightarrow FT' \\ & & T' &\rightarrow *FT'/\epsilon \\ & & F &\rightarrow (E)/id. \end{aligned}$$

First look at first and fill.
If we have ϵ , then look for follow.

$$\begin{aligned} \text{first}(E) &= \{ (, id \} & \text{follow}(E) &= \{ \$,) \} \\ \text{first}(E') &= \{ +, \epsilon \} & \text{follow}(E') &= \{ \$,) \} \\ \text{first}(T) &= \{ (, id \} & \text{follow}(T) &= \{ +,), \$ \} \\ \text{first}(T') &= \{ *, \epsilon \} & \text{follow}(T') &= \{ +,), \$ \} \\ \text{first}(F) &= \{ (, id \} & \text{follow}(F) &= \{ +, *,), \$ \} \end{aligned}$$

Input symbol (terminals in tie to)

Non-terminals	(	id	+	*	\$	)
E	$E \rightarrow TE'$	$E \rightarrow TE'$				
E'			$E' \rightarrow +E'$		$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$	$T \rightarrow FT'$				
T'			$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow (E)$	$F \rightarrow id$				

→ This is LL(1) grammar

2.

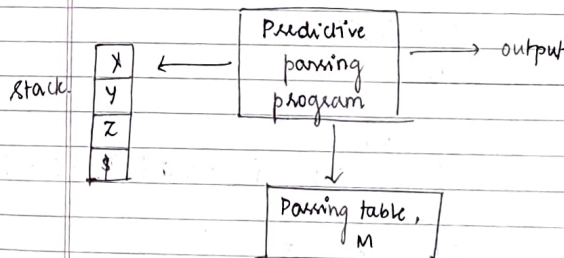
N-T	id	;	int	real	\$
D	$D \rightarrow L; T$				
L	$L \rightarrow idL'$				
L'	$L' \rightarrow idL'$ $L' \rightarrow \epsilon$				
T			$T \rightarrow int$	$T \rightarrow real$	

→ This is not LL(1) grammar.

$$3. S \rightarrow iEtSS$$

⇒ Non-recursive predicting parsing:

□ □ □ a / + b / \$



1) $id + id * id$

$E \rightarrow TE'$

$E' \rightarrow +TE' | \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' | \epsilon$

$F \rightarrow (E) | id$

Look at parsing table for substitution.

By default start state $\leftarrow$

Matched	Stack	Input	Action
	$E \$$	$id + id * id$	
	$TE' \$$	$id + id * id$	$E \rightarrow TE'$
	$FT'E' \$$	$id + id * id$	$T \rightarrow FT'$
id	$T'E' \$$	$id + id * id$	$F \rightarrow id$
id	$T'E' \$$	$+ id * id$	
id	$E' \$$	$+ id * id$	$T' \rightarrow \epsilon$
id	$E' \$$	$+ id * id$	
$id +$	$TE' \$$	$id * id$	$E' \rightarrow +TE'$
$id +$	$FT'E' \$$	$id * id$	$T \rightarrow FT'$
$id + id$	$T'E' \$$	$id * id$	$F \rightarrow id$
$id + id *$	$FT'E' \$$	id	$T' \rightarrow *FT'$
$id + id * id$	$T'E' \$$	$*$	$F \rightarrow id$
$id + id * id$	$T'E' \$$		$T' \rightarrow \epsilon$
$id + id * id$	$E' \$$		$E' \rightarrow \epsilon$

2) $S \rightarrow A$ Input: $(id * id)$.

$S \rightarrow (S * A)$

$A \rightarrow id$

First (S) = $\{ (, id \}$

First (A) = $\{ id \}$

Follow (S) = $\{ \$, * \}$

Follow (A) = $\{ follow(S) \} = \{ \$, * \} \cup \{) \} = \{ \$, *,) \}$

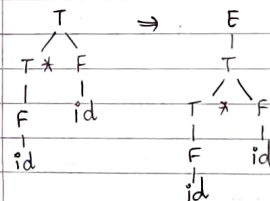
N-T	(	id	\$	*	)
S	$S \rightarrow (S * A)$	$S \rightarrow A$			
A		$A \rightarrow id$			

Matched Stack Input Action

	$S \$$	$id * id$	
	$A \$$	$id * id$	$S \rightarrow A$
	id		
$($	$S * A \$$	$(id * id)$	$S \rightarrow (S * A)$
$($	$A * A \$$	$id * id$	$S \rightarrow A$
$(id$	$A * A \$$	$id * id$	$A \rightarrow id$
$(id *$	$A \$$	id	
$(id * id$	$id \$$	id	$A \rightarrow id$
$(id * id)$	$\$$	$*$	

$\Rightarrow$ Bottom-up parsing:

$id * id \Rightarrow F * id \Rightarrow T * id \Rightarrow T * F \Rightarrow$



$E \rightarrow E * T / T$
 $T \rightarrow T * F / F$
 $F \rightarrow (E) / id$

$\Rightarrow$ Handle parsing:

A handle is a substring that matches the body of a production and whose reduction represents one step along the reverse of a rightmost derivation.

Right sentential form	Handle	Reducing production
$id * id$	id	$F \rightarrow id$
$F * id$	F	$T \rightarrow F$
$T * id$	id	$F \rightarrow id$
$T * F$	$T * F$	$T \rightarrow T * F$
T	T	$E \rightarrow T$
E		

2) $S \rightarrow OSI/OI$; Right sentential : 00001111

Right sentential form	Handle	Reducing production
00001111	01	$S \rightarrow 01$
000SI111	OSI	$S \rightarrow OSI$
00S111	OSI	$S \rightarrow OSI$
0SI11	OSI	$S \rightarrow OSI$
SI11	OSI	$S \rightarrow OSI$
S		

3) $S \rightarrow SS^+ / SS^* / a$; Right sentential : $SS^+ a a^* a^+$

Right sentential form	Handle	Reducing production
$SS^+ a a^* a^+$	SS^+	$S \rightarrow SS^+$
$SA^* a^+$	a	$S \rightarrow a$
$SS^* a^+$	SS^*	$S \rightarrow SS^*$
SA^+	a	$S \rightarrow a$
SS^+	SS^+	$S \rightarrow SS^+$
S		

2) $S \rightarrow OSO/ISI/2$ Input : 10201

Stack	Input	Action
\$	10201\$	Shift '1'
\$1	0201\$	Shift '0'
\$10	201\$	Shift '2'
\$102	01\$	Reduce $S \rightarrow 2$
\$10S	01\$	Shift '0'
\$10SO	1\$	Reduce $S \rightarrow OSO$
\$1S	1\$	Shift '1'
\$1SI	\$	Reduce $S \rightarrow ISI$
\$S	\$	Accept.

⇒ Conflicts during shift-reduce parsing:
1. Shift/reduce conflicts.
2. Reduce/reduce conflicts.

Imp ⇒ Shift reduce parser:
Shift some symbols of input to the stack until
a reduction can be applied.

$E \rightarrow E+T/T$ $T \rightarrow T * F / F$ $F \rightarrow (E) / id$	1) Stack	Input	Action
	\$	id * id \$	Shift 'id'
	\$id	* id \$	Reduce $F \rightarrow id$
	\$idF	* id \$	Reduce $T \rightarrow F$
	\$T	* id \$	Shift '*'
	\$T*	id \$	Shift 'id'
	\$T*id	\$	Reduce $F \rightarrow id$
	\$T*F	\$	Reduce $T \rightarrow T * F$
	\$T	\$	Reduce $E \rightarrow T$
	\$E	\$	Accept.

$A \rightarrow AC/B$
 $A \rightarrow BA$
 $A' \rightarrow AA'/\epsilon$

Revision (Mod-2)

classmate

Date _____

Page _____

1. ~~Q1~~ Consider a grammar

$$E \rightarrow E - E$$

$$E \rightarrow E * E$$

$$E \rightarrow id$$

Input string: $id - id * id$. Perform shift reduce parsing.

2. $S \rightarrow (L) | a$

$$L \rightarrow L, S / S$$

I/P string: (a, a)

a) Construct predictive parse table.

b) Perform non-recursive predictive parsing.